E-Commerce Platform

Full-Stack Web Application

Date: May 28, 2026

FastAPI + React + PostgreSQL + Docker

Table of Contents

1 Project Overview

2 Tech Stack

3 Project Structure

4 Backend - Data Models

5 Backend - API Endpoints

6 Backend - Authentication & Authorization

7 Frontend - Routes & Pages

8 Frontend - State Management

9 Frontend - Component Architecture

10 Setup & Installation

11 Running the Application

12 Deployment

1. Project Overview

The E-Commerce Platform is a full-stack web application that provides a complete online shopping experience. It includes user authentication, product catalog browsing, shopping cart management, order processing, payment simulation, and an administrative dashboard for managing products, orders, and users.

The project is structured as a monorepo containing two main sub-projects: the backend API (ecommerce-api) built with FastAPI, and the frontend SPA (ecommerce-frontend) built with React. They communicate via a RESTful JSON API secured with JWT tokens.

Key Features

- User registration and JWT-based authentication
- Product catalog with category filtering and pagination
- Shopping cart with add, update, and remove items
- Order creation from cart contents
- Payment simulation (initiate, process, complete)
- Role-based access control: regular users and administrators
- Admin dashboard with statistics, product CRUD, order management, and user listing
- Responsive design with Tailwind CSS
- Docker Compose for one-command infrastructure setup

2. Tech Stack

Backend

Technology	Purpose
FastAPI	Python web framework for building REST APIs
SQLAlchemy 2.0	Async ORM for database interactions
PostgreSQL 15	Relational database
Alembic	Database migration management
Redis 7	Caching and session storage
Docker & Docker Compose	Containerization and orchestration
python-jose	JWT token encoding and decoding
passlib / bcrypt	Password hashing and verification

Frontend

Technology	Purpose
React 18	UI library for building component-based interfaces
Vite 4	Fast build tool and development server
React Router v6	Client-side routing with nested layouts
Tailwind CSS 3	Utility-first CSS framework
Zustand	Lightweight state management for auth
TanStack React Query	Server state management and caching
Axios	HTTP client with interceptors

3. Project Structure

The monorepo is organized into two top-level directories:

```
ecommerce/
+-- ecommerce-api/            # Backend (FastAPI)
|   +-- app/
|   |   +-- api/              # Route handlers
|   |   +-- core/             # Config, security, dependencies
|   |   +-- db/               # SQLAlchemy models, migrations
|   |   +-- models/           # Pydantic schemas
|   |   +-- services/         # Business logic layer
|   |   +-- main.py           # FastAPI app entry
|   +-- tests/
|   +-- Dockerfile
|   +-- docker-compose.yml
|   +-- requirements.txt
|
+-- ecommerce-frontend/       # Frontend (React)
    +-- src/
    |   +-- api/              # API client modules
    |   +-- components/       # Reusable components
    |   |   +-- layout/       # Navbar, Footer, Layout
    |   |   +-- guards/       # ProtectedRoute, AdminRoute
    |   |   +-- ui/           # Button, Input, Card, etc.
    |   |   +-- product/      # ProductCard, ProductGrid
    |   |   +-- cart/         # CartItem
    |   +-- pages/            # Route page components
    |   |   +-- public/       # Home, Login, Register, Catalog
    |   |   +-- user/         # Cart, Orders, Profile
    |   |   +-- admin/        # Dashboard, Products, Orders, Users
    |   +-- store/            # Zustand auth store
    |   +-- App.jsx           # Router configuration
    +-- index.html
    +-- vite.config.js
    +-- tailwind.config.js
    +-- package.json
```

4. Backend - Data Models

The database schema consists of five main entities. Below is a brief description of each:

Entity	Fields
User	id (UUID), email (unique), password_hash, role (user/admin), created_at, updated_at
Product	id (UUID), name, description, price (float), stock (int), category, images (JSON), is_active, timestamps
Cart	id (UUID), user_id (FK unique), created_at
CartItem	id (UUID), cart_id (FK), product_id (FK), quantity (int)
Order	id (UUID), user_id (FK), items (JSON), total (float), status (enum: pending/paid/shipped/delivered/cancelled), timestamps
Payment	id (UUID), order_id (FK), amount (float), method (string), status (enum: pending/processing/succeeded/failed), transaction_id, created_at

All tables use UUID primary keys. Relationships are enforced via foreign keys. Orders store a snapshot of their items as JSON to preserve history even if products change.

5. Backend - API Endpoints

All endpoints are prefixed with /api/v1. Authentication is handled via JWT Bearer tokens.

Authentication

Method	Route	Description
POST	/auth/register	Register a new user with email and password
POST	/auth/login	Login and receive a JWT access token

Users

Method	Route	Description
GET	/users/me	Get the authenticated user's profile
PUT	/users/me	Update the authenticated user's email

Products

Method	Route	Description
GET	/products/	List products (optional: ?category=, ?skip=, ?limit=)
GET	/products/{id}	Get a single product by ID
POST	/products/	Create a new product (admin only)
PUT	/products/{id}	Update a product (admin only)
DELETE	/products/{id}	Delete a product (admin only)

Cart

Method	Route	Description
GET	/cart/	Get the current user's cart with items
POST	/cart/items	Add a product to the cart
PUT	/cart/items/{id}	Update item quantity
DELETE	/cart/items/{id}	Remove item from cart
DELETE	/cart/	Clear all items from cart

Orders

Method	Route	Description
GET	/orders/	List the current user's orders
GET	/orders/{id}	Get a specific order
POST	/orders/	Create an order from the current cart contents

PATCH	/orders/{id}/status	Update order status (admin only)

Payments

Method	Route	Description
POST	/payments/	Initiate a payment for an order
GET	/payments/{id}	Get payment details
POST	/payments/{id}/simulate	Simulate payment processing (development only)

Admin

Method	Route	Description
GET	/admin/stats	Get platform statistics (order count, revenue, users, products)
GET	/admin/users	List all users
GET	/admin/orders	List all orders

6. Authentication & Authorization

The backend uses JWT (JSON Web Tokens) for stateless authentication. When a user logs in, the server validates their credentials and returns a signed JWT containing the user ID, email, and role. The token is valid for 30 minutes by default (configurable via ACCESS_TOKEN_EXPIRE_MINUTES in .env).

Auth Flow

- User submits credentials via POST /auth/login
- Server verifies email and password against the database (passlib/bcrypt)
- Server creates a JWT with payload: {user_id, email, role, exp}
- Client stores the token in localStorage
- All subsequent authenticated requests include Authorization: Bearer <token>
- Server decodes and validates the token on each request via the get_current_user dependency
- Admin-only endpoints additionally check that user.role == 'admin'

Security Measures

- Passwords are hashed with bcrypt before storage
- JWT secret key is configured via environment variable
- Tokens have a configurable expiration time
- Role-based access control on admin endpoints
- HTTP 401 is returned for missing or invalid tokens
- HTTP 403 is returned for non-admin users accessing admin routes

7. Frontend - Routes & Pages

The frontend uses React Router v6 with nested layouts and route guards.

Route	Page	Description	Access
/	Home	Featured products, welcome section	Public
/login	Login	Email/password sign-in form	Public
/register	Register	New account creation form	Public
/products	Catalog	Product grid with category filter	Public
/products/:id	ProductDetail	Full product info + add to cart	Public
/cart	Cart	View/edit cart items, place order	Auth
/checkout	Checkout	Payment initiation (redirect from order)	Auth
/orders	Orders	List of user's orders	Auth
/orders/:id	OrderDetail	Single order with items + pay button	Auth
/profile	Profile	Update email address	Auth
/admin	Dashboard	Stats cards (orders, revenue, users, products)	Admin
/admin/products	AdminProducts	Product CRUD table with form modal	Admin
/admin/orders	AdminOrders	All orders table with status badges	Admin
/admin/users	AdminUsers	All users table with role badges	Admin

Route Guards

ProtectedRoute checks that a valid JWT token exists in the auth store. If not present, users are redirected to /login. AdminRoute additionally checks that the user's role is 'admin'. Non-admin users are redirected to the homepage.

8. Frontend - State Management

The frontend uses two approaches for state management:

Zustand (Client State)

The auth store manages authentication state. It persists the user object and JWT token to localStorage so the session survives page reloads. The store exposes setAuth() and logout() actions which update both memory and localStorage.

TanStack React Query (Server State)

All data fetched from the API (products, cart, orders, admin stats) is managed by TanStack React Query. This provides automatic caching, background refetching, loading and error states, and optimistic updates via the queryClient. Each API module has corresponding hooks that use useQuery for reads and useMutation for writes.

Axios Interceptor

A centralized Axios client attaches the JWT token from localStorage to every request. A response interceptor catches 401 errors and redirects to the login page, ensuring that expired or invalid tokens result in a clean re-authentication flow.

9. Frontend - Component Architecture

The frontend follows a component-based architecture with clear separation of concerns:

Layout Components

- Navbar: Role-aware navigation (public links, user links, admin link), login/logout buttons
- Footer: Simple copyright footer
- Layout: Wraps all routes with Navbar + Footer + main content area (Outlet)

UI Components (Reusable)

- Button: Variants for primary, secondary, danger, ghost actions
- Input: Labeled form input with error state display
- Card: Consistent container with shadow and border
- Spinner: Centered loading animation
- Badge: Color-coded status indicator (pending, paid, shipped, cancelled, etc.)

Feature Components

- ProductCard / ProductGrid: Display products in a responsive card grid
- CartItem: Individual cart item with quantity controls and remove button

Route Guards

- ProtectedRoute: Redirects unauthenticated users to /login
- AdminRoute: Redirects non-admin users to /

10. Setup & Installation

Prerequisites

- Docker Desktop (recommended) for running the backend infrastructure
- Node.js 18+ and npm for the frontend
- Python 3.10+ (if running backend without Docker)
- Git for version control

Backend Setup (Docker - Recommended)

```
    cd ecommerce-api
docker compose up -d

# The API will be available at http://localhost:8000
# PostgreSQL runs on port 5432, Redis on port 6379
```

Backend Setup (Manual)

```
    cd ecommerce-api
python -m venv venv
venv\Scripts\activate
pip install -r requirements.txt

# Copy .env.example to .env and edit database credentials
# Make sure PostgreSQL and Redis are running locally

uvicorn app.main:app --reload --port 8000
```

Frontend Setup

```
    cd ecommerce-frontend
npm install
npm run dev

# The dev server will start at http://localhost:5173
# API calls are proxied to http://localhost:8000
```

11. Running the Application

Once both the backend and frontend are running, follow these steps to test the full flow:

1. Open http://localhost:5173 in your browser.
2. Click 'Register' to create a new account.
3. Browse the product catalog under 'Products'.
4. Click on a product to view details and click 'Add to Cart'.
5. Navigate to 'Cart' to see your items. Adjust quantities or remove items.
6. Click 'Place Order' to create an order from your cart.
7. On the order detail page, click 'Pay Now' to initiate and simulate payment.
8. View all your orders under 'Orders'.

Admin access:
To access the admin panel, manually set a user's role to 'admin' in the database, or use the following SQL command:

```
UPDATE users SET role = 'admin' WHERE email = 'your@email.com';
```

Default Admin Credentials

There are no default admin accounts. After registering a new user, promote the account to admin via direct database update. The admin panel is accessible at http://localhost:5173/admin.

12. Deployment

The application is container-ready. Each component can be deployed independently or together via Docker Compose.

Option A: Docker Compose (single server)

```
# On your server:
git clone <repo-url>
cd ecommerce-api
docker compose up -d

# Build and serve the frontend:
cd ../ecommerce-frontend
npm install
npm run build

# Serve dist/ with nginx or any static server
```

Option B: Separate services

- Deploy the FastAPI app on any PaaS (Heroku, Railway, Fly.io, etc.)
- Use a managed PostgreSQL service (Supabase, AWS RDS, etc.)
- Deploy the frontend build to Vercel, Netlify, or Cloudflare Pages
- Set VITE_API_URL environment variable on the frontend to point to your deployed API

Environment Variables

Key environment variables for production deployment:

```
# Backend (.env)
DATABASE_URL=postgresql+asyncpg://user:pass@host:5432/db
REDIS_URL=redis://host:6379
JWT_SECRET_KEY=<generate-a-strong-random-secret>
JWT_ALGORITHM=HS256
ACCESS_TOKEN_EXPIRE_MINUTES=60

# Frontend (.env)
VITE_API_URL=https://your-api-domain.com/api/v1
```